

FUNCTIONS



Variables

- 📖 Local Variables
- 📖 Global Variables
- 📖 Dynamic SQL
- 📖 What Are T-SQL Expressions?
- 📖 Control-of-flow Statements

What Are T-SQL Variables?

A Transact-SQL local variable is an object that can hold a single data value of a specific type

Variables in batches and scripts are typically used to:

- Count the number of times a loop is performed
- Hold data to be tested by a control-of-flow statement
- Save data values to be returned by a function return value

```
DECLARE @local_variable as <data_type>
```

```
declare @food varchar(20)  
set @food = 'ice cream'  
...  
WHERE Description = @food
```



Local Variable

- 📖 A local variable is a named location in memory defined by a user that stores a value
- 📖 Null is the Initial Value for local variable
- 📖 Typical uses for a local variable
 - 📋 To facilitate repeated use of constant values
 - 📋 To perform conditional branching in Transact-SQL code
 - 📋 To return custom messages to the client that contain variable information
 - 📋 To pass information to and from a stored procedure
 - 📋 To avoid using a subquery

Rules for Local Variables

- 📖 Variable names must start with “@”
- 📖 Variables must have a system or user-defined datatype
 - 📋 Any properties associated with the user-defined datatype are not inherited
 - 📋 Any rules or defaults bound to a user-defined datatype are not inherited
- 📖 Variables are local to the batch, stored procedure, or trigger in which they are declared
 - 📋 When the batch, stored procedure, or trigger terminates, the variable is dropped

Declaring Local Variables

Syntax:

```
declare      variable_name datatype  
             [, variable_name      datatype ...]
```

Example:

```
declare      @myqty      int,  
             @myid       char(4)
```

 Variables must be declared before they can be used

 When declared, the value of a local variable is set to NULL

Viewing Local Variable Values

 Simplified syntax:

`select variable_name`

 Example:

```
declare @mynumber          int
select @mynumber
```

`-`

`NULL`

 This is the same **select** that is used to query data from tables

Assigning Values to Local Variables

Three methods

-  During Declaration
-  Assignment **select** using an expression
-  Assignment **select** using a table value
-  Assignment **update**

During Declaration

Example1:

```
declare @x int=1
```

```
declare @x int=(Select avg(salary) from Instructor)
```

Example2:

```
declare @x int
```

```
Set @x=1
```

Assignment select and Expressions

Simplified syntax:

```
select variable_name = expression  
    [, variable_name = expression ...]
```

Examples:

```
declare    @number int,  
            @copy int,  
            @sum int  
  
select     @number = 10  
  
select     @copy = @number,  
            @sum = @number + 100
```

This **select** is an “assignment **select**”

 No information is returned to the user

Assignment select and Table Values

Simplified syntax:

```
select variable_name = column_name  
from table_name  
[where condition]
```

Examples:

```
declare @AD_id char(11)  
select @AD_id = au_id  
      from authors  
      where au_fname = "Ann" and au_lname = "Dull"
```

This **select** is an assignment **select**

 No information is returned to the user

If the **select** returns multiple values, only the last value remains in the variable

Assignment update

Simplified syntax:

```
update table_name
set {column_name | variable_name} = expression
    [, {column_name | variable_name} = expression ... ]
[where condition]
```

Examples:

```
declare @pub_name varchar(40)
update publishers
    set      city = "Escanaba",
           state = "MI",
           @pub_name = pub_name
    where pub_id = "0736"
```

 No information is returned to the user

 If the **update** modifies multiple rows, only the last assigned value remains in the variable

Restrictions on Local Variables

- 📖 Variables can be used only in place of constants
- 📖 If an assignment **select** returns no values, the variable value remains unchanged
- 📖 If an assignment **select** returns multiple values, the variable retains the last value

Global Variable

- 📖 A global variable is a named location in memory, defined and maintained by Adaptive Server
- 📖 0 is the initial value for @@error variable
- 📖 Rules for global variables
 - 📋 Names start with “@@”
 - 📋 Cannot be created by users
 - 📋 Cannot be assigned values by users
 - 📋 Value assignment can be local to the server or to the connection

Common Global Variables

@@rowcount

-  Returns the number of rows affected by the last statement

@@error

-  Returns the error number generated by the last statement

@@identity

-  Returns the value last inserted into an IDENTITY column

@@version

-  Returns the version number of the server

What Are T-SQL Expressions?

Symbols and operators evaluated to obtain a single value

Expressions can be combined if one of these is true:

- The expressions have the same data type
- The data type with the lower precedence can be converted to the data type with the higher precedence

```
SELECT ProductID, Variable_Name + 2
```


Control-of-flow Statements

These are the control-of-flow keywords:

- BEGIN...END

- GOTO

- IF...ELSE

- RETURN

- BREAK

- CONTINUE

- WHILE

- WAITFOR

Control of Flow Examples

What Are Batch Directives?

- These control movement within a T-SQL file

```
IF @cost <= @compareprice
BEGIN
    PRINT 'These products can be purchased for less than
    $'+RTRIM(CAST(@compareprice AS varchar(20)))+'. '
END
ELSE
    PRINT 'The prices for all products in this category exceed
    $'+ RTRIM(CAST(@compareprice AS varchar(20)))+'. '
GO
```

Case Expression

Evaluates a list of conditions and returns one of multiple possible result expressions.

The CASE expression has two formats:

- 📖 The simple CASE expression compares an expression to a set of simple expressions to determine the result.
- 📖 The searched CASE expression evaluates a set of Boolean expressions to determine the result.

Both formats support an optional ELSE argument.

Case Expression Cont.

Syntax

Simple CASE expression:

CASE input_expression

 WHEN when_exp THEN result_exp[..n] [ELSE
 else_result_exp]

END

Searched CASE expression:

CASE

 WHEN Boolean_exp THEN result_exp[...n] [ELSE else_result_exp]

END

Case Examples

Message Statements

 Three statements can return text information to users:

 **select**

 **print**

 **raiserror**

select

- 📖 Useful for returning information in the result set

- 📖 Simplified syntax:

```
select { "user_message" | variable_name | column_list }
```

- 📖 Example:

```
select "The average price is too low."
```

```
select "The average price is ", @avg_price
```

```
select "The average price is ", avg(price)  
                                from titles
```

- 📖 Can return text, variable values, and table data in any combination

print with Strings and Variables

- 📖 Useful for returning debugging messages

- 📖 Simplified syntax:

```
print { "user_message" | variable_name }
```

- 📖 Examples:

```
print "There is no user by that name."
```

```
print @temp_title_id
```

- 📖 Variables used directly by **print** (as in the second example) must be *char* or *varchar*

print with Argument Lists

- 📖 Useful for returning debugging messages

- 📖 Simplified syntax:

```
print "user_message" , argument_list
```

- 📖 Example:

```
print "Table %1! is not owned by user %2!",  
      @table_name, @user_name
```

- 📖 Placeholders indicate where arguments are inserted

- 📋 Must be in the format of %*n*! where:

- *n* is an integer 1 – 20

- Values of *n* are used sequentially

- 📖 Arguments can be any datatype except *text* or *image*

- 📋 Arguments are temporarily converted to *char*

raiserror

- 📖 Useful for returning error messages in the message window
- 📖 Example:

```
raiserror 70500 "Table %1! not found.", @mytable
```
- 📖 **raiserror** returns two pieces of information
 - 📋 Error number, which is stored in *@@error*
 - 📋 User-defined error message, which is returned to the user
- 📖 Error messages may or may not be reusable, depending on where they are specified
 - 📋 Specified only in the **raiserror** statement – not reusable
 - 📋 Specified in the *sysusermessages* table – reusable

Returning Nonreusable Error Messages

Simplified Syntax:

```
raiserror error_number {"error_message" | variable } [, argument_list ]
```

Examples:

```
raiserror 30250 "The title does not exist."
```

```
raiserror 30250 @no_title_text
```

```
raiserror 70500 "Table %1! not found.", @mytable
```

```
raiserror 70500 @no_table_text, @mytable
```

 Message number must be 20000 or greater

 If a message is not stored in *sysusermessages*, then it must be stated every time it is used

Creating Reusable Error Messages

Simplified add syntax:

`sp_addmessage message_number, "error_message"`

Example:

`sp_addmessage 30250, "The title does not exist."`

`sp_addmessage 70500, "Table %1! not found."`

Simplified drop syntax:

`sp_dropmessage message_number`

Example:

`sp_dropmessage 30250`

 Error messages must be 20000 or greater

 Error messages can include %*n*! placeholders

Returning Reusable Error Messages

Simplified syntax:

```
raiserror error_number [, argument_list ]
```

Examples:

```
raiserror 30250
```

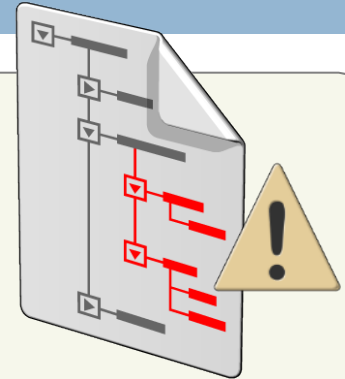
```
raiserror 70500, @mytable
```

-  Message number must be 20000 or greater
-  There must be a message defined in *sysusermessages* for that number
-  If the message in *sysusermessages* contains placeholders, you must include an argument list

Structured Exception Handling

TRY/CATCH

```
BEGIN TRY
    -- Generate divide-by-zero error.
    SELECT 1/0;
END TRY
BEGIN CATCH
    -- Execute error retrieval routine.
    EXECUTE usp_GetErrorInfo;
END CATCH;
```



RAISERROR

```
RAISERROR (N'This is message %s %d.', -- Message text.
    10, -- Severity,
    1, -- State,
    N'number', -- First argument.
    5); -- Second argument.
-- The message text returned is: This is message number 5.
GO
```

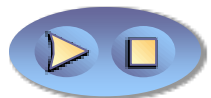
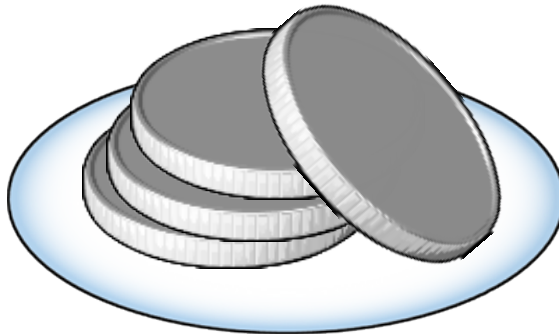
Overview of Transactions

- 📖 Transaction Fundamentals
- 📖 Transactions and the Database Engine
- 📖 Basic Transaction Statement Definitions
- 📖 What are Transaction Isolation Levels?
- 📖 Using Nested Transactions

Transaction Fundamentals

A Transaction:

- Is a sequence of operations performed as a single logical unit of work
- Exhibits the four ACID Properties
 - Atomicity – must be an atomic unit of work
 - Consistency - must leave all data in a consistent state
 - Isolation - must be isolated from the modifications made by any other concurrent transactions
 - Durability – persists even after system failure



Role of Transactions

- 📖 Protect data from software, hardware, and power failures
- 📖 Allow for data isolation so that multiple users can access data simultaneously without interfering with one another

Transactions and the Database Engine

The Database Engine provides:

- Locking facilities that preserve transaction isolation
 - Transaction Isolation Levels control when locks are taken and how long they are held
- Logging facilities that ensure transaction durability
 - Write-ahead log (WAL) guarantees no data modifications are written before they are logged
 - Checkpoints write records to a data file and contain lists of all active transactions
- Transaction management features that enforce transaction atomicity and consistency
 - Transactions must be successfully completed or their modifications are undone

[transaction video](#)

Basic Transaction Statement Definitions

BEGIN TRANSACTION

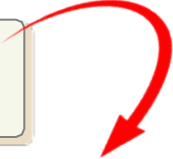
```
BEGIN { TRAN | TRANSACTION } [  
Transaction_name | @tran_name_variable ]
```



```
BEGIN TRAN T1;  
UPDATE table1 ...;
```

COMMIT TRANSACTION

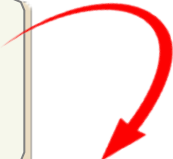
```
COMMIT { TRAN | TRANSACTION } [ transaction_name |  
@tran_name_variable ]
```



```
COMMIT TRAN T1;
```

ROLLBACK TRANSACTION

```
ROLLBACK { TRAN | TRANSACTION } [ transaction_name |  
@tran_name_variable | savepoint_name |  
@savepoint_variable ]
```



```
ROLLBACK TRAN T1;
```



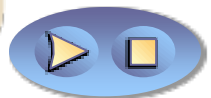
Using Nested Transactions

- Explicit transactions can be nested to support transactions in stored procedures

```
CREATE TABLE TestTrans(Cola INT PRIMARY KEY,  
                        Colb CHAR(3) NOT NULL);  
GO  
CREATE PROCEDURE TransProc @PriKey INT, @CharCol CHAR(3) AS  
BEGIN TRANSACTION InProc  
INSERT INTO TestTrans VALUES (@PriKey, @CharCol)  
INSERT INTO TestTrans VALUES (@PriKey + 1, @CharCol)  
COMMIT TRANSACTION InProc;  
GO  
BEGIN TRANSACTION OutOfProc; /* Starts a transaction */  
GO  
EXEC TransProc 1, 'aaa';  
GO  
ROLLBACK TRANSACTION OutOfProc; /* Rolls back the outer  
                                transaction */  
  
GO  
EXECUTE TransProc 3, 'bbb';  
GO  
SELECT * FROM TestTrans;  
GO
```

Cola Colb

1	bb
2	bb



@@trancount

- 📖 *@@trancount* is a global variable that keeps track of the nesting level
 - 📋 **begin tran** increments *@@trancount* by 1
 - 📋 **commit tran** decrements *@@trancount* by 1
 - 📋 **rollback tran** sets *@@trancount* to 0
 - 📋 **save tran** and **rollback savepoint_name** do not change *@@trancount*

Introducing Functions

- 📖 Types of Functions
- 📖 What Is a Scalar Function?
- 📖 What Is an Inline Table-Valued Function?
- 📖 What Is a Multi-Statement Table-Valued Function?

What Are User-Defined Functions?

A User-Defined Function is a routine that accepts parameters, performs an action, and returns the result of that action as a value.

Benefits of using User-Defined Functions

- Modular programming for reusable logic.
- Complex operations can be optimized for faster execution.
- Logic performed in database reduces network traffic

What Are T-SQL Functions?

Functions	Notes
Rowset	Return objects that can be used as table references
Examples: CONTAINSTABLE, OPENDATASOURCE, OPENQUERY	
Aggregate	Operate on a collection but returns a single value
Examples: AVG, CHECKSUM_AGG, SUM, COUN	
Ranking	Return a ranking value for each row in a partition
Examples: RANK, DENSE_RANK	
Scalar	Operate on a single value and then return a single value
Examples: CREATE FUNCTION dbo.ufn_CubicVolume	

Types of Functions

Types of Functions

ü Scalar Functions

ü Inline Table-Valued Functions

ü Multi-Statement Table-Valued Functions

ü Built-in Functions

How To Implement Different Types Of User-Defined Functions

Types	Usage
Scalar-valued	• Scalar is specified in the RETURNS clause • Can be defined with multiple T-SQL statements
Inline table-valued	• TABLE is specified in the RETURNS clause • Does not have associated return variables • <i>select_stmt</i> is the single SELECT statement that defines the return value
Multi-statement table-valued	• TABLE is specified in the RETURNS clause • <i>function_body</i> is used as a series of T-SQL statements that populate a TABLE return variable • <i>@return_variable</i> is used to store and accumulate rows that are returned as the value

What Is a Scalar Function?

Scalar Functions:

- ü Return a single data value
- ü Can be either inline or multi-statement
- ü Can return any data type except for text, ntext, image, cursor, and timestamps

```
CREATE FUNCTION [ schema_name. ] function_name  
( [ { @parameter_name [ AS ] [ type_schema_name. ] parameter_data_type  
    [ = default ] [ READONLY ] }  
    [ ,...n ]  
]  
)  
RETURNS return_data_type
```

What Is an Inline Table-Valued Function?

Inline Table-Valued Function:

- ü Returns a TABLE data-type
- ü Has no function body
- ü Is comprised of a single result set

```
CREATE FUNCTION [ schema_name. ] function_name  
( [ { @parameter_name [ AS ] [ type_schema_name. ] parameter_data_type  
    [ = default ] [ READONLY ] }  
    [ ,...n ]  
]  
)  
RETURNS TABLE
```

What Is a Multi-Statement Table-Valued Function?

Multi-statement Table-Valued Function:

- ü Returns a TABLE data-type
- ü Has a function body defined by BEGIN and END blocks
- ü Defines a table-type variable and schema
- ü Inserts rows from multiple Transact-SQL statements into the returned table

```
CREATE FUNCTION [ schema_name. ] function_name  
( [ { @parameter_name [ AS ] [ type_schema_name. ] parameter_data_type  
    [ = default ] [ READONLY ] }  
    [ ,...n ] ] )  
RETURNS @return_variable TABLE <table_type_definition>
```

Working with Functions

- 📖 Deterministic and Nondeterministic Functions
- 📖 Guidelines for Creating Functions
- 📖 Rewriting Stored Procedures as Functions

Deterministic and Nondeterministic Functions

Type	Description
Deterministic Functions	Always return the same result given a specific input
Nondeterministic functions	May return different results given a specific input
Built-in functions	Are deterministic or nondeterministic depending on how they are applied

Determines whether a user defined function is deterministic or not

```
SELECT  
[IsDeterministic] = objectproperty(object_id('dbo.myUDF'),  
'IsDeterministic')
```

Guidelines for Creating Functions



Determine function type



Create one function for one task



Create, test, and troubleshoot



Qualify object names inside function



Consider ability of SQL Server 2008 to index function results

Restrictions When Creating User-Defined Functions

You cannot use user-defined functions to:

- Update data
 - Use stored procedures instead
- Define or create new objects in the database
 - Objects referred to by the function have to be previously declared and created
- Perform transactions

Rewriting Stored Procedures as Functions

Convert your stored procedure to a function if:

- ü It is expressible as a single SELECT statement
- ü It does not perform update operations
- ü It does not require a dynamic EXECUTE statement
- ü It only returns one result set
- ü Its primary purpose is to build intermediate results

Performance Considerations For Using User-Defined Functions

```
SELECT MyCalculation(column_name)
FROM table_name
WHERE MyCondition(column_name)
```

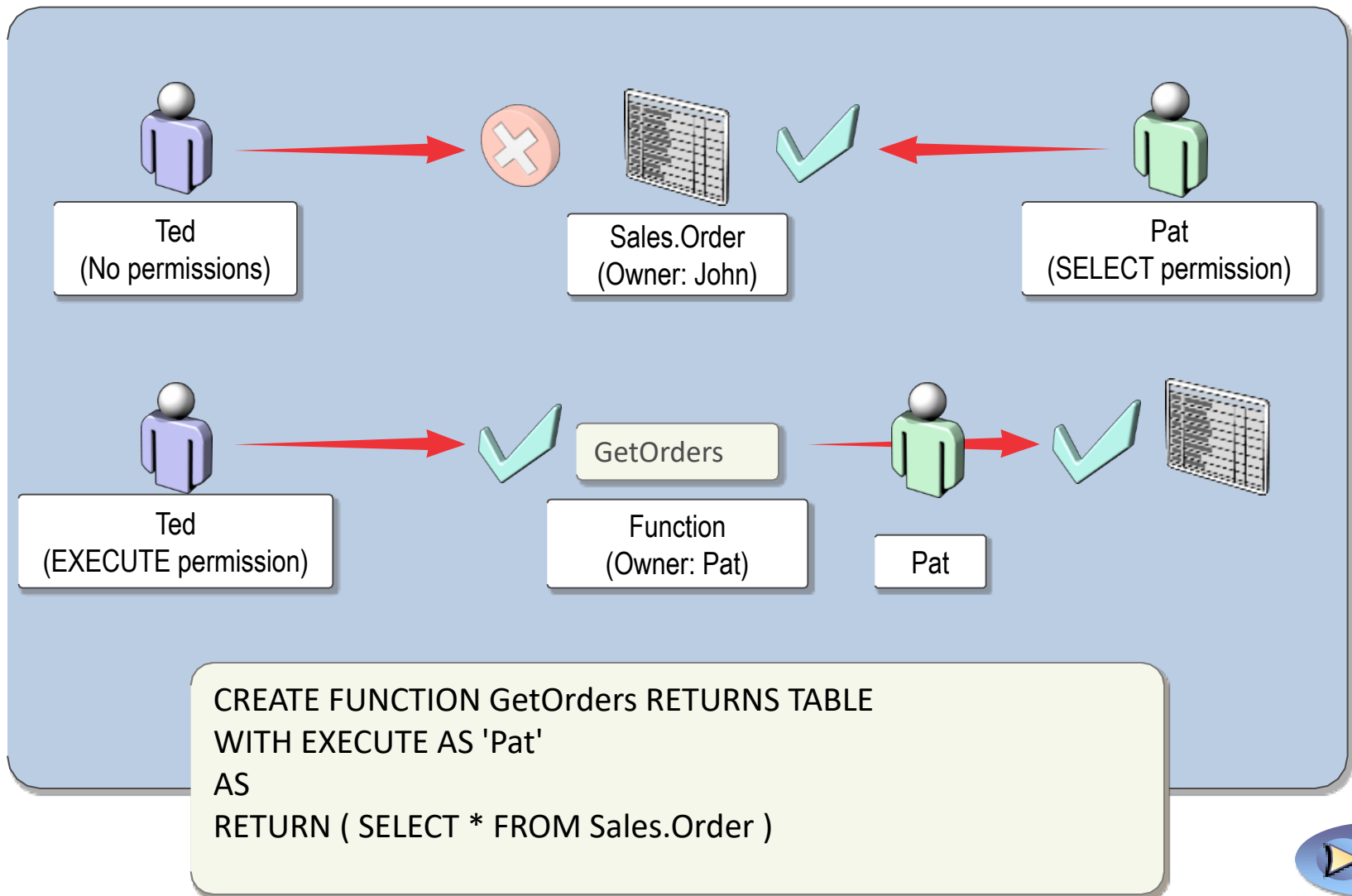
- Both functions will be called once for each row in the table.
- If both functions require 0.1 second for each execution, the query will require:
 - 1 second for 5 rows
 - 10 seconds for 50 rows
 - 1 hour for 18,000 rows

Controlling Execution Context



- What Is Execution Context?
- The EXECUTE AS Clause
- Extending Impersonation Context

What Is Execution Context?



The EXECUTE AS Clause

The Execute AS Clause:

- ü Enables Impersonation
- ü Provides access to modules via impersonation
- ü Can be used to impersonate server-level principals or logins via the EXECUTE AS LOGIN statement
- ü Can be used to impersonate database level principals or users via the EXECUTE AS USER statement

```
CREATE FUNCTION GetOrders RETURNS TABLE
WITH EXECUTE AS { CALLER | SELF | OWNER | 'user_name' }
AS
RETURN ( SELECT * FROM Sales.Order )
```

Dynamic SQL

- 📖 Introducing Dynamic SQL
- 📖 Using Dynamic SQL
- 📖 Considerations for Using Dynamic SQL

Static SQL versus Dynamic SQL

Static SQL

-  SQL whose text is specified when the application, script, or stored procedure is created
-  The keywords and the referenced objects are fixed at the time the statements are written
-  All of the examples of SQL statements shown in this course so far have been static SQL

Dynamic SQL

-  SQL whose text is specified at execution time
-  The keywords and the referenced objects can be determined at the time the statement is executed

Introducing Dynamic SQL

Dynamic SQL

- Allows query to be built using variables
- Places query into variable

```
SET @SQLString = N'SELECT @SalesOrderOUT =  
MAX(SalesOrderNumber)  
FROM Sales.SalesOrderHeader  
WHERE CustomerID = @CustomerID';
```


Using Dynamic SQL

Using sp_executesql

```
sp_executesql [ @stmt = ] stmt  
[  
    {, [@params=] N'@parameter_name data_type [ OUT | OUTPUT ][,...n]' }  
    {, [ @param1 = ] 'value1' [ ,...n ] }  
]
```

Using EXECUTE

```
[ { EXEC | EXECUTE } ]  
{ [ @return_status = ]  
  { module_name [ ;number ] | @module_name_var }  
  [ [ @parameter = ] { value | @variable [ OUTPUT ]  
    | [ DEFAULT ] } ] [ ,...n ] [ WITH RECOMPILE ] } [;]
```

Executing Transact-SQL Dynamically

Syntax:

```
exec[ute] ( { string_constant | char_variable }  
          [ + { string_constant | char_variable } ] [ + ... ] )
```

Example:

```
declare      @table_name  varchar (30),  
             @col_list    varchar (30)  
  
select      @table_name = "publishers",  
            -- @table_name content is decided at Run time  
            @col_list = "pub_name, state"  
            -- @col_name content is decided at Run time  
  
exec  ("select " + @col_list + " from " +  
       @table_name)  
  
go
```



Restrictions on Dynamic SQL

- 📖 Variables used in the **execute** statement must be *char* or *varchar*
- 📖 Statements that cannot be executed dynamically
 - 📋 **exec** or **execute**
 - 📋 **use**
 - 📋 Transaction control statements
 - 📋 **create table** statements that create temporary tables
- 📖 Outside of the **execute** statement, you cannot use a variable where a keyword, object name, or table column is syntactically required
 - 📋 For example, you cannot execute the following because a variable is being used where a table name is required:

```
select * from @table_name
```

Considerations for Using Dynamic SQL

Security Considerations:

- ü SQL Injection
- ü Security is checked for every object

Performance Considerations:

- ü Query plan not reused frequently
- ü Use `sp_executesql` for best chance at plan reuse
- ü Performance related to complexity of query